

Теория параллелизма

Отчет

Задание №6-7

Выполнил, Пономарев Андрей Александрович 23934
25.05.2025

Цель работы: реализовать программу для решения уравнения теплопроводности, изучить время выполнения программы на CPU на одном ядре и на нескольких, изучить время выполнения на GPU, выполнить профилирование программы с использованием “Nsight Systems”.

Используемый компилятор: gpc++

Используемый профилировщик: Nsight Systems

Замер времени: библиотека chrono

Выполнение на CPU

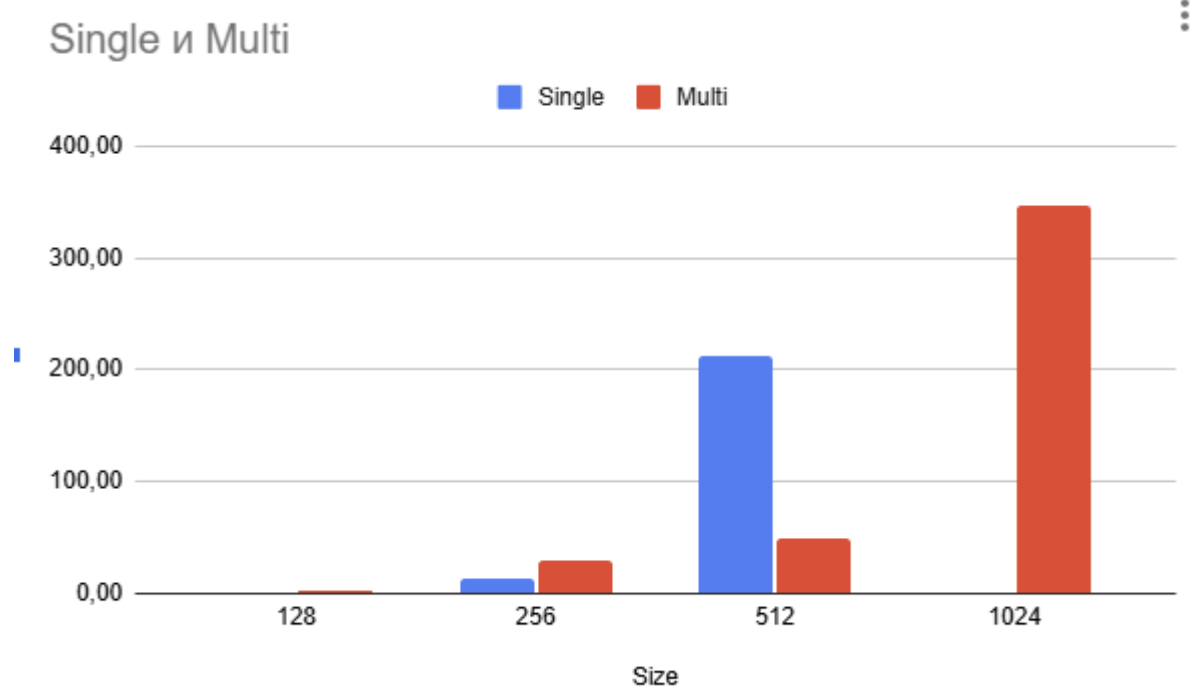
CPU-onecore

Размер сетки	Время выполнения	Точность (final error)	Количество итераций
128*128	0.920166	0.000001	30075
256*256	13.858765	0.000001	102885
512*512	211.502202	0.000001	339599

CPU-multicore

Размер сетки	Время выполнения	Точность	Количество итераций
128*128	2.446336	0.000001	30075
256*256	29.769453	0.000001	102885
512*512	49.666926	0.000001	339599
1024*1024	346.737076	0.000001	1000000

Диаграмма сравнения время работы CPU-one и CPU-multi



Выполнение на GPU

Этапы оптимизации на сетке 512*512

Этап №	Время выполнения	Точность	Максимальное количество итераций	Комментарии
1	15.71309	0.000001	1000000	первая версия
2	15.631527	0.000001	1000000	попытка внедрить collapse()
3	12.019444	0.000001	1000000	error считается не на каждом шаге

Начальный вариант:

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Style	Range
98.1	503252831	1	503252831.0	503252831.0	503252831	503252831	0.0	PushPop	init
1.0	4876794	1	4876794.0	4876794.0	4876794	4876794	0.0	PushPop	while
0.5	2546345	100	25463.5	22427.5	21609	258820	23632.4	PushPop	calc
0.4	2276244	100	22762.4	22147.0	21353	30312	1449.1	PushPop	copy

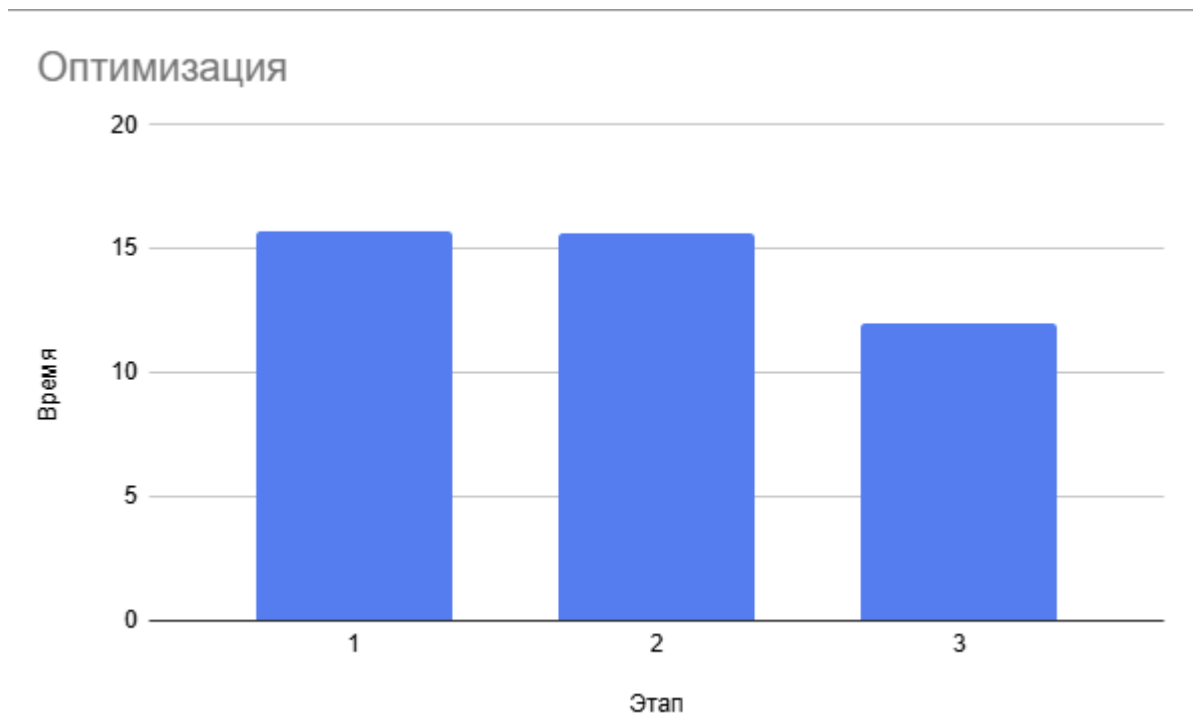
Второй этап: внедрение collapse():

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Style	Range
98.3	500165503	1	500165503.0	500165503.0	500165503	500165503	0.0	PushPop	init
0.8	4267137	1	4267137.0	4267137.0	4267137	4267137	0.0	PushPop	while
0.4	2219715	100	22197.2	20084.5	19484	215165	19509.9	PushPop	calc
0.4	2001112	100	20011.1	19841.5	19366	30053	1104.4	PushPop	copy

Третий этап: Проверка ошибки каждую 1000 итерацию

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Style	Range
98.5	565215828	1	565215828.0	565215828.0	565215828	565215828	0.0	PushPop	init
0.8	4304648	1	4304648.0	4304648.0	4304648	4304648	0.0	PushPop	while
0.4	2249081	100	22490.8	19989.5	19233	251441	23162.8	PushPop	calc
0.3	2004417	100	20044.2	19669.0	18857	30433	1707.1	PushPop	copy

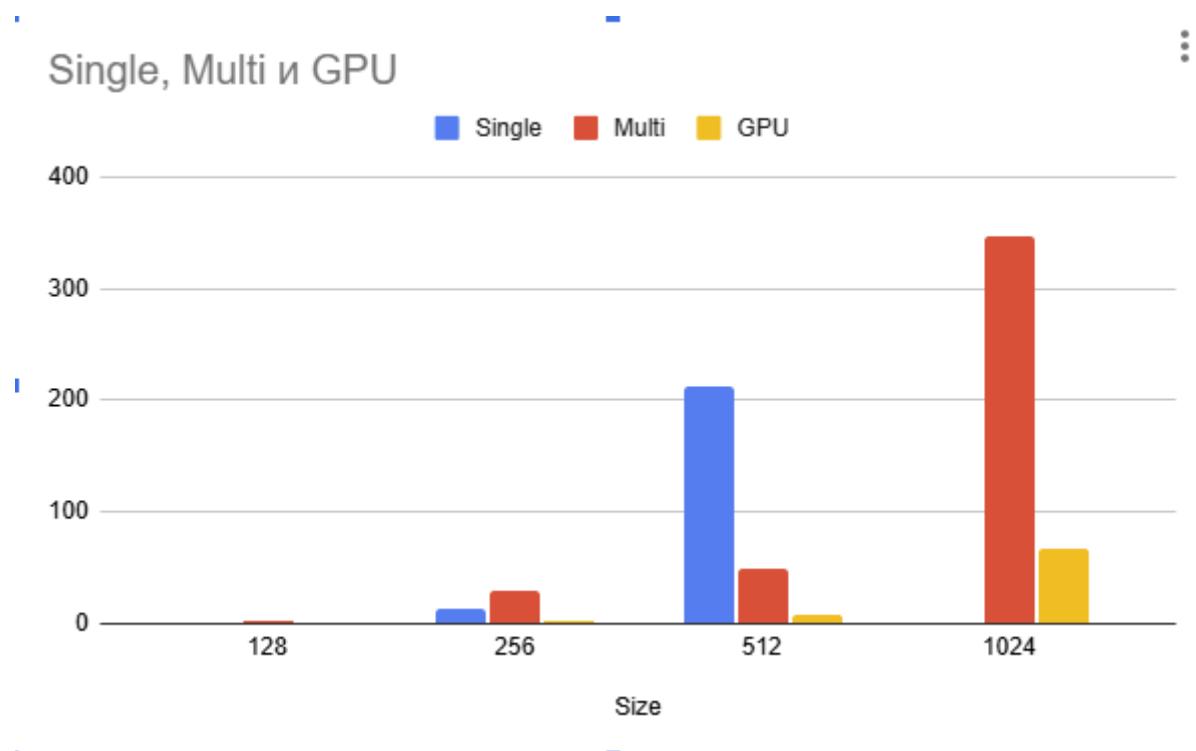
Диаграмма оптимизации



GPU – оптимизированный вариант

Размер сетки	Время выполнения	Точность	Количество итераций
128*128	0.647406	0.000001	31000
256*256	2.290811	0.000001	103000
512*512	8.616552	0.000001	340000
1024*1024	66.571557	0.000001	999999

Диаграмма сравнения времени работы CPU-one, CPU-multi, GPU(оптимизированный вариант) для разных размеров сеток



Выполнение на GPU с cuBLAS

Оптимизация на GPU с cuBLAS

Этап №	Время выполнения	Точность	Максимальное количество итераций	Комментарии
1	8.56879	9.92418e-07	1000000	первая версия
2	12.1728	13.5956	1000000	упрощенное копирование
3	7.85133	9.92418e-07	1000000	объединение вычисления новой сетки и расчета ошибки

Первый этап: изначальная версия

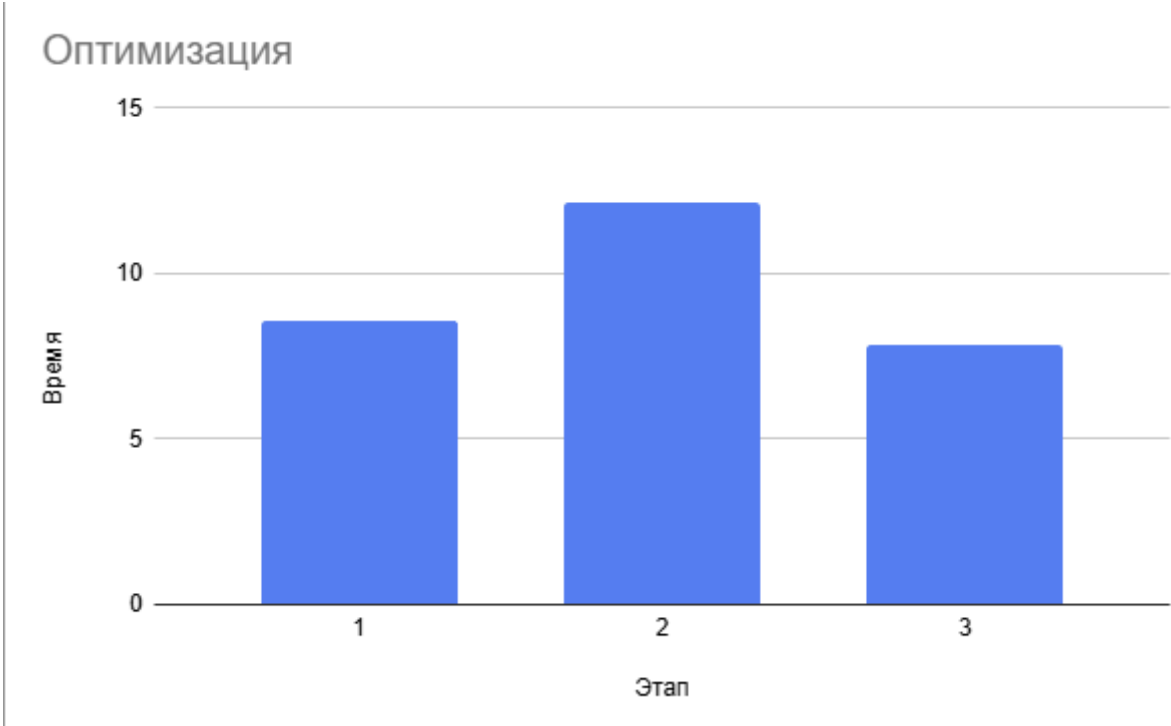
Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Style	Range
49.4	16825456	1	16825456.0	16825456.0	16825456	16825456	0.0	PushPop	SolverLoop
24.3	8264112	1	8264112.0	8264112.0	8264112	8264112	0.0	PushPop	InitGrid
12.0	4081851	100	40818.5	39203.0	38219	145446	10878.1	PushPop	Compute
11.6	3957627	100	39576.3	39239.0	37754	50814	1786.8	PushPop	CopyBack
2.6	901194	1	901194.0	901194.0	901194	901194	0.0	PushPop	ErrorCalc

Второй этап: упрощенное копирование

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Style	Range
48.4	7452367	1	7452367.0	7452367.0	7452367	7452367	0.0	PushPop	SolverLoop
17.9	2751055	1	2751055.0	2751055.0	2751055	2751055	0.0	PushPop	InitGrid
14.0	2149785	100	21497.9	19933.5	19115	162116	14310.3	PushPop	Compute
13.0	2003924	100	20039.2	19603.5	18777	37111	2355.3	PushPop	CopyBack
6.8	1053308	1	1053308.0	1053308.0	1053308	1053308	0.0	PushPop	ErrorCalc

Третий этап: объединение вычисления новой сетки и расчета ошибки

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Style	Range
51.9	8057132	1	8057132.0	8057132.0	8057132	8057132	0.0	PushPop	SolverLoop
14.7	2279324	1	2279324.0	2279324.0	2279324	2279324	0.0	PushPop	InitGrid
14.1	2196959	100	21969.6	20337.5	19699	122652	10465.9	PushPop	Compute+Error
13.0	2017005	100	20170.1	19503.0	18938	39194	2834.8	PushPop	CopyBack
6.4	988261	1	988261.0	988261.0	988261	988261	0.0	PushPop	ErrorReduce



оптимизированный вариант

Размер сетки	Время выполнения	Точность	Количество итераций
128	0.599365	8.77535e-07	30501
256	2.2255	9.91175e-07	103001
512	7.85133	9.92418e-07	340001
1024	68.9399	1.37251e-06	1000000

Вывод

На итоговой диаграмме видно, что распараллеливание на GPU работает быстрее всего. Большое количество физических ядер в совокупности большим объёмом видеопамяти позволяют добиться реального распределения работы в этой задаче. Также хорошо видно, что чем больше сетка, тем быстрее работает GPU по сравнению с остальными. Это объясняется тем, что данные на GPU сразу уходят на обработку, а на CPU необходимо ожидать “своей очереди”. На небольших же значениях разрыв меньше. Это объясняется тем, что хотя процессор имеет небольшое количество ядер, его хватает для распараллеливания, соответственно время “ожидания в очереди” сокращается. К тому же процессорные ядра быстрее, соответственно если данных на обработку не так много, процессор в некоторых задачах может даже обгонять видеокарту.